Algoritma Genetika

NAMA	: Ihsanta Zaki Sanjaya
KELAS	: 2 Teknik Komputer B
NRP	: 3223600058
DOSEN	: Riyanto Sigit ST, M.Kom, Ph.D
MATA KULIAH	: Praktikum Sistem Cerdas
TANGGAL	: 21 Mei 2025

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2025

Praktikum 11

Algoritma Genetika

A. Tujuan

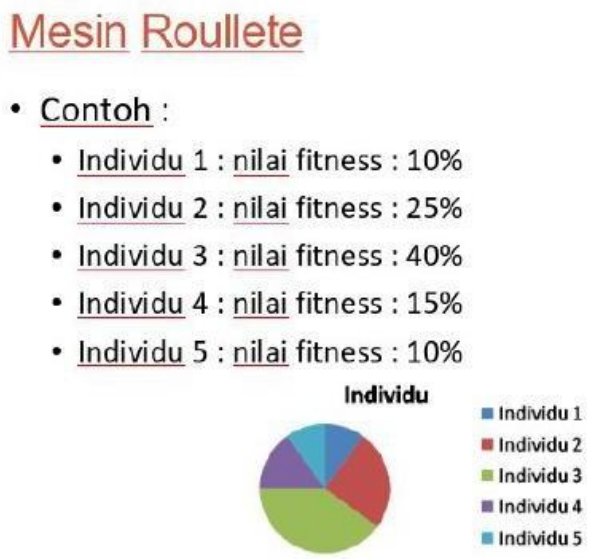
1. Mahasiswa memahami konsep dasar algoritma genetika.
2. Mahasiswa mampu mengimplementasikan algoritma genetika dalam bentuk program.

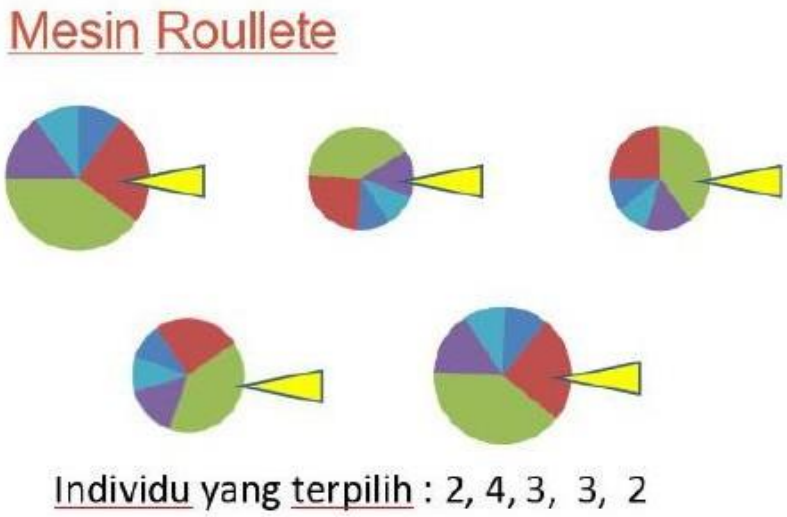
B. Dasar Teori

Algoritma Genetika merupakan algoritma pencarian yang didasarkan pada mekanisme seleksi alam dan genetika. Algoritma ini meniru proses evolusi biologis, di mana individu-individu mengalami perubahan genetik untuk beradaptasi dengan lingkungan mereka. Dalam konteks komputasi, algoritma genetika digunakan untuk mencari solusi optimal dari suatu permasalahan dengan cara yang terinspirasi dari seleksi alam, yaitu individu yang "kuat" akan bertahan. Algoritma genetika pertama kali dikembangkan oleh John Holland pada tahun 1975.

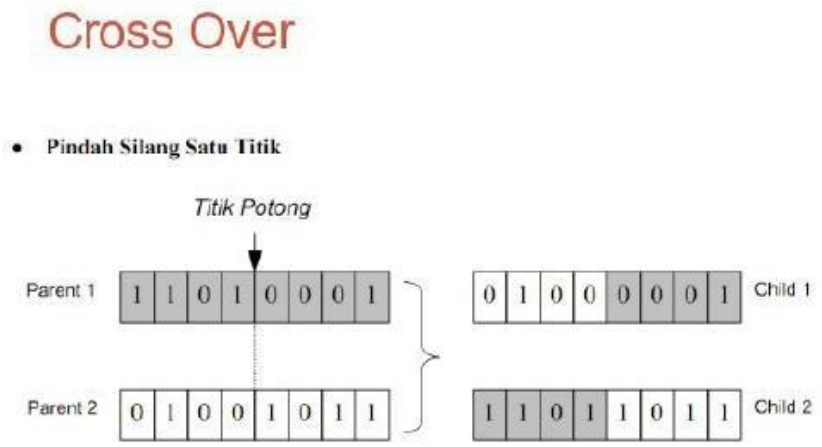
Dalam algoritma genetika, terdapat beberapa terminologi penting yang perlu dipahami. Populasi adalah kumpulan dari individu-individu yang merepresentasikan solusi dari masalah yang ingin dipecahkan. Setiap individu dalam populasi merupakan sebuah solusi tunggal, yang terdiri dari genome. Genome adalah kumpulan gen yang terdapat dalam suatu individu. Kromosom adalah struktur data yang merepresentasikan gen-gen dari individu tersebut, dan umumnya direpresentasikan sebagai rangkaian biner. Gen sendiri adalah bagian dari kromosom yang mengkodekan parameter tertentu dari solusi. Kualitas dari sebuah solusi (individu) dinilai dengan menggunakan nilai fitness, yang mengindikasikan seberapa baik solusi tersebut dalam memecahkan masalah.

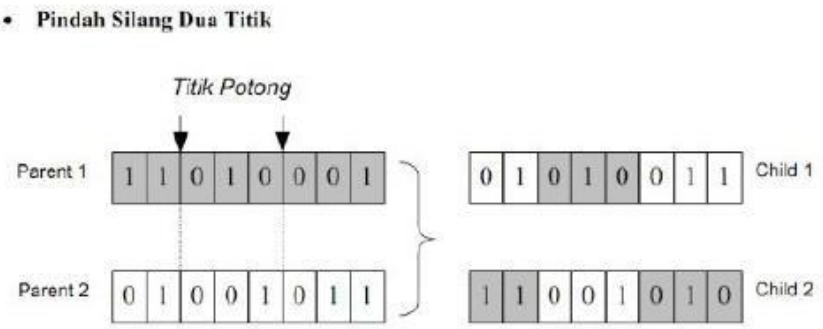
Prosedur dasar algoritma genetika melibatkan beberapa tahapan. Pertama, populasi awal dibangkitkan secara acak. Kemudian, dilakukan evaluasi fitness untuk setiap individu dalam populasi. Selanjutnya, individu-individu dipilih berdasarkan nilai fitness-nya dalam proses seleksi. Individu dengan nilai fitness yang lebih tinggi memiliki peluang lebih besar untuk terpilih. Salah satu metode seleksi yang umum digunakan adalah metode roulette wheel, di mana setiap individu mendapatkan bagian proporsional dalam "roda roulette" berdasarkan nilai fitness-nya.





Setelah proses seleksi, dilakukan crossover (pindah silang) dan mutasi pada individu-individu yang terpilih untuk menghasilkan populasi baru. Crossover adalah operator genetik yang menggabungkan informasi genetik dari dua induk untuk menghasilkan keturunan. Proses ini melibatkan pertukaran gen secara acak antara dua induk. Salah satu jenis crossover adalah pindah silang satu titik, di mana gen dari induk ditukar mulai dari satu titik potong. Jenis lainnya adalah pindah silang dua titik, di mana terdapat dua titik potong yang menentukan bagian gen yang ditukar. Mutasi adalah operator genetik yang mengubah nilai satu atau lebih gen dalam kromosom. Mutasi bertujuan untuk menjaga keragaman populasi dan mencegah konvergensi prematur. Proses ini dilakukan dengan memberikan nilai inversi atau menggeser nilai gen pada gen yang dipilih untuk dimutasi.





Mutasi

Proses seleksi, crossover, dan mutasi diulangi hingga kriteria penghentian terpenuhi, seperti mencapai jumlah iterasi maksimum atau menemukan solusi dengan nilai fitness yang memuaskan.

C. Percobaan

Contoh Aplikasi

Contoh: memaksimalkan angka 1 dalam string yang panjangnya 5, terdiri dari 1 dan 0

⇒**Populasi dengan 4 individual , masing-masing punya 5 gen pada kromosomnya. Nilai gen ditentukan secara random**

***population size* = 4** ***chromosome length* = 5** ***fitness function* = jumlah gen 1**

Populasi Awal

individual 1:
chromosome = 11001
fitness = 3

individual 2:
chromosome = 00001
fitness = 1

individual 3:
chromosome = 11111
fitness = 5

individual 4:
chromosome = 01110
fitness = 3

Kromosom ditentukan secara acak !

Seleksi Roullete

Current Population:
i1: 11001, 3
i2: 00001, 1
i3: 11111, 5
i4: 01110, 3

Total Fitness
TF = 3+1+5+3 = 12

Probability of each individual being selected:
prob(i1) = 3/12 = 0.25
prob(i2) = 1/12 = 0.08
prob(i3) = 5/12 = 0.42
prob(i4) = 3/12 = 0.25

Assume:
Roullete dilakukan 4 kali dan menghasilkan 1 copy dari i1, 2 copy dari i3, 1 copy dari i4

Pemasangan Kromosom

Current mating pool:
mate 1: 11001 *(i1)*
mate 2: 11111 *(i3)*
mate 3: 11111 *(i3)*
mate 4: 01110 *(i4)*

Assume:
Pasangan (secara acak) :
(mate 1, mate 3)
dan
(mate 2, mate 4)

Pairs:

Pair 1:	Pair 2:
11001	11111
11111	01110

Cross Over

p_c=1.0

for pair 1:
cross-over site: 3
110 | 01 ● 11011
111 | 11 ● 11101

for pair 2:
cross-over site: 1
1 | 1111 ● 11110
0 | 1110 ● 01111

the new individuals:
i1: 11011 i3: 11110
i2: 11101 i4: 01111

Mutasi

i1: 11011 ● 01011

i3: 11110 ● 11100

Populasi Baru

individual 1:
chromosome =01011
fitness = 3

individual 2:
chromosome =11101
fitness = 4

individual 3:
chromosome =11100
fitness = 3

individual 4:
chromosome =01111
fitness = 4

Program :

```
#include <stdlib.h>
#include <conio.h>
#include <stdio.h>

int POP_SIZE = 4;
int SIZE = 5;
int individu[4][5];
int individuBaru[4][5];
int fitnessValue[5];
int totalFitness, bestFitness;
double percentValue[4];

void init_population() {
    for (int i = 0; i < POP_SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            int r = rand() % 2;
            individu[i][j] = r;
        }
    }
}

int fitness() {
    int jum;
    totalFitness = 0;

    for (int i = 0; i < POP_SIZE; i++) {
        jum = 0;
        for (int j = 0; j < SIZE; j++) {
            jum = jum + individu[i][j];
        }
        fitnessValue[i] = jum;
        totalFitness = totalFitness + jum;
    }

    int max = 0;
    for (int i = 0; i < POP_SIZE; i++) {
        if (max < fitnessValue[i]) max = fitnessValue[i];
    }

    bestFitness = max;

    for (int i = 0; i < POP_SIZE; i++) {
        printf("Individu[%d]: ", (i + 1));
        for (int j = 0; j < SIZE; j++) {
            printf("%d", individu[i][j]);
        }
        printf(" NF:%d\n", fitnessValue[i]);
    }
```

```
    printf("Total Fitness: %d Best Fitness: %d\n", totalFitness, bestFitness);
    return bestFitness;
}

// Selection
void rouletteSelection() {
    double totalPercentFitness = 0.0;
    int k;

    for (int j = 0; j < POP_SIZE; j++) {
        percentValue[j] = (fitnessValue[j] * 100.0) / totalFitness;
        printf("Percent individu[%d]: %.2f\n", (j + 1), percentValue[j]);
        totalPercentFitness += percentValue[j];
    }

    for (int i = 0; i < POP_SIZE; i++) {
        double daerahPersen = 0;
        double random = rand() % 100;

        for (k = 0; k < POP_SIZE; k++) {
            daerahPersen += percentValue[k];
            if (random <= daerahPersen) {
                break;
            }
        }

        for (int j = 0; j < SIZE; j++) {
            individuBaru[i][j] = individu[k][j];
        }
    }

    for (int i = 0; i < POP_SIZE; i++) {
        printf("Individu[%d]: ", (i + 1));
        for (int j = 0; j < SIZE; j++) {
            individu[i][j] = individuBaru[i][j];
            printf("%d", individu[i][j]);
        }
        printf("\n");
    }
}

// Cross Over
void crossOver() {
    for (int i = 0; i < POP_SIZE; i += 2) {
        int r = rand() % 5;
        printf("random: %d\n", r);

        for (int j = 0; j < SIZE; j++) {
            if (j >= r) {
                individuBaru[i][j] = individu[i + 1][j];
                individuBaru[i + 1][j] = individu[i][j];
            } else {
                individuBaru[i][j] = individu[i][j];
```

```
                individuBaru[i + 1][j] = individu[i + 1][j];
            }
        }
    }

    for (int i = 0; i < POP_SIZE; i++) {
        printf("Individu[%d]: ", (i + 1));
        for (int j = 0; j < SIZE; j++) {
            individu[i][j] = individuBaru[i][j];
            printf("%d", individu[i][j]);
        }
        printf("\n");
    }
}

// Mutasi
void mutasi() {
    for (int i = 0; i < POP_SIZE; i += 2) {
        int r = rand() % 5;
        printf("random: %d\n", r);

        for (int j = 0; j < SIZE; j++) {
            individuBaru[i][j] = individu[i][j];
        }

        if (individu[i][r] == 0)
            individuBaru[i][r] = 1;
        else
            individuBaru[i][r] = 0;
    }

    for (int i = 0; i < POP_SIZE; i++) {
        printf("Individu[%d]: ", (i + 1));
        for (int j = 0; j < SIZE; j++) {
            individu[i][j] = individuBaru[i][j];
            printf("%d", individu[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int best;
    init_population();

    for (int iter = 0; iter < 100; iter++) {
        best = fitness();
        if (best == 5) break;

        rouletteSelection();
        crossOver();
        mutasi();
    }
```

```
    return 0;
```

```
PS D:\1. KULIAH\Semester IV\3. Sistem Cerdas\Demo Akhir\output> & .\'genetika.exe'
Individu[1]: 11001 NF:3
Individu[2]: 00000 NF:0
Individu[3]: 11111 NF:5
Individu[4]: 11010 NF:3
Total Fitness:11 Best Fitness:5
Percent individu[1]: 27.27
Percent individu[2]: 0.00
Percent individu[3]: 45.45
Percent individu[4]: 27.27
Individu[1]: 11010
Individu[2]: 11001
Individu[3]: 11001
Individu[4]: 11111
random: 2
random: 2
Individu[1]: 11001
Individu[2]: 11010
Individu[3]: 11111
Individu[4]: 11001
random: 1
random: 1
Individu[1]: 10001
Individu[2]: 11010
Individu[3]: 10111
Individu[4]: 11001
Individu[1]: 10001 NF:2
Individu[2]: 11010 NF:3
Individu[3]: 10111 NF:4
Individu[4]: 11001 NF:3
Total Fitness:12 Best Fitness:4
Percent individu[1]: 16.67
Percent individu[2]: 25.00
Percent individu[3]: 33.33
Percent individu[4]: 25.00
Individu[1]: 11010
Individu[2]: 11001
Individu[3]: 10111
Individu[4]: 11010
random: 1
random: 3
Individu[1]: 11001
Individu[2]: 11010
Individu[3]: 10110
Individu[4]: 11011
random: 4
random: 2
Individu[1]: 11000
Individu[2]: 11010
Individu[3]: 10010
Individu[4]: 11011
Individu[1]: 11000 NF:2
Individu[2]: 11010 NF:3
Individu[3]: 10010 NF:2
Individu[4]: 11011 NF:4
Total Fitness:11 Best Fitness:4
```

Tugas :

Algoritma Genetika Untuk Mencari Nama Secara Acak

- Sebuah kata ditentukan sebagai target, misalnya: 'BASUKI'. Bila setiap huruf diberi nilai dengan nilai urut alfabet, maka targetnya bisa dinyatakan sebagai besaran numerik:

  Target=[2 1 18 21 11 9]
- Komputer akan membangkitkan katadengan jumlah huruf yang sama dengan target secara acak, terus-menerus hingga diperoleh kata yang sama dengan kata target.

Definisi Individu dan Fitness

- Individu adalah satu kata yang muncul dari proses acak tersebut, misalnya: AGHSQE atau [1 7 8 19 17 5]
- Satu individu mempunyai n gen integer yang setiap gennya menyatakan no urut alfabet.
- Nilai fitness adalah inversi dari perbedaan antara nilai kata yang muncul (individu) dan target yang ditentukan. Misalnya kata yang muncul: AGHSQE dan targetnya BASUKI maka, nilai perbedaannya:

  E = |1-2| + |7-1| + |8-18| + |19-21| + |17-11| + |5-9|
  = 1+6+10+2+6+4 = 29

  Fitness = (26)(6) -29 = 156-29 = 127
- Fitness didefinisikan:

  $fitness\ (k) = 156 - \sum | g\ (k)_n - t_n |$

Dimana gn adalah gen ke n dari individu ke k dan gen ke n dari target.

Pembangkitan Populasi Awal

Populasi awal dibangkitkan dengan cara membangkitkan semua huruf dalam sejumlah kata (individu) yang dibangkitkan.

```
14 20  9  1 17  3  -- N T I A Q C   >> Fitness = 83
 2  5 18  5  6  6  -- B E R E F F   >> Fitness = 127
 8  5 15 15 24  6  -- H E O O X F   >> Fitness = 120
 5 22 14 11 19 23  -- E V N K S W   >> Fitness = 95
19 19  8  6 19  7  -- S S H F S G   >> Fitness = 85
20 16  3 21  8 10  -- T P C U H J   >> Fitness = 103
19 13 12 23 15 10  -- S M L W O J   >> Fitness = 113
15 23  4 16  6 17  -- O W D P F Q   >> Fitness = 88
 5  1  6 19 21 18  -- E A F S U R   >> Fitness = 119
10 12 18  6 17  8  -- J L R F Q H   >> Fitness = 114
10  1  2  8  6 19  -- J A B H F S   >> Fitness = 103
21 18 21 24 26 19  -- U R U X Z S   >> Fitness = 90
```

Seleksi, Cross-Over, dan Mutasi

- Seleksi dilakukan dengan menggunakan roulette-wheel.
- Cross-over, dilakukan dengan menukar gen-gen terpilih antar dua induk, seperti pada gen biner.

```
8  5 15 15 24  6  -- H E O O K F
5 22 14 11 19 23  -- E V N K S W

8  5 14 11 19  6  -- H E N K S F
5 22 15 15 24 23  -- E V O O K W
```

- Mutasi dilakukan dengan mengacak kembali nilai 1-26 dari gen yang dimutasikan.

```
8  5 14 11 19  6  -- H E N K S F

8  5 19 11 19  6  -- H E S K S F
```

Hasil Algoritma Genetika

```
5  3 19 19 14 18  --- E C S S N R   >> Fitness = 137
5  3 19 19 14 10  --- E C S S N J   >> Fitness = 145
5  3 19 19 14 10  --- E C S S N J   >> Fitness = 145
5  3 19 19 10 10  --- E C S S J J   >> Fitness = 147
5  3 19 24 11 10  --- E C S X K J   >> Fitness = 147
5  2 19 19 11 10  --- E B S S K J   >> Fitness = 149
5  2 19 22 10 10  --- E B S V J J   >> Fitness = 149
5  2 19 22 11 10  --- E B S V K J   >> Fitness = 150
5  2 19 22 11 10  --- E B S V K J   >> Fitness = 150
5  2 19 21 11 10  --- E B S U K J   >> Fitness = 151
5  2 19 21 11 10  --- E B S U K J   >> Fitness = 151
5  2 19 21 11 10  --- E B S U K J   >> Fitness = 151
1  2 19 21 11 10  --- A B S U K J   >> Fitness = 153
2  2 19 21 11 10  --- B B S U K J   >> Fitness = 154
2  2 19 21 11 10  --- B B S U K J   >> Fitness = 154
2  2 19 21 11  9  --- B B S U K I   >> Fitness = 155
2  2 19 21 11  9  --- B B S U K I   >> Fitness = 155
2  2 19 21 11  9  --- B B S U K I   >> Fitness = 155
2  2 19 21 11  9  --- B B S U K I   >> Fitness = 155
2  1 19 21 11  9  --- B A S U K I   >> Fitness = 156
```

Program :

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define GEN_LENGTH 6
#define POP_SIZE 100
#define MAX_FITNESS 156
#define MUTATION_RATE 0.1

char target[] = "BASUKI";
char population[POP_SIZE][GEN_LENGTH + 1];
char newPopulation[POP_SIZE][GEN_LENGTH + 1];
```

```c
int fitness[POP_SIZE];
int totalFitness, bestFitness;

// Calculate fitness of an individual
int calculateFitness(const char *individual) {
    int fit = MAX_FITNESS;
    for (int i = 0; i < GEN_LENGTH; i++) {
        int diff = abs((individual[i] - 'A' + 1) - (target[i] - 'A' + 1));
        fit -= diff;
    }
    return fit;
}

// Evaluate the fitness of the entire population
int evaluatePopulation() {
    int bestIndividualIndex = 0;
    totalFitness = 0;

    for (int i = 0; i < POP_SIZE; i++) {
        fitness[i] = calculateFitness(population[i]);
        totalFitness += fitness[i];

        if (fitness[i] > fitness[bestIndividualIndex]) {
            bestIndividualIndex = i;
        }
    }

    bestFitness = fitness[bestIndividualIndex];

    printf(" ");
    for (int j = 0; j < GEN_LENGTH; j++) {
        printf("%2d ", population[bestIndividualIndex][j] - 'A' + 1);
    }
    printf("--- %s >> Fitness = %d\n", population[bestIndividualIndex],
fitness[bestIndividualIndex]);

    return bestIndividualIndex;
}

// Initialize the population with random strings
void initializePopulation() {
    for (int i = 0; i < POP_SIZE; i++) {
        for (int j = 0; j < GEN_LENGTH; j++) {
            population[i][j] = 'A' + rand() % 26;
        }
        population[i][GEN_LENGTH] = '\0';
    }
}

// Select two parents based on fitness (simple selection of top 2)
void selection(int *parent1, int *parent2) {
    *parent1 = 0;
    *parent2 = 1;
```

```
    for (int i = 0; i < POP_SIZE; i++) {
        if (fitness[i] > fitness[*parent1]) {
            *parent2 = *parent1;
            *parent1 = i;
        } else if (fitness[i] > fitness[*parent2] && i != *parent1) {
            *parent2 = i;
        }
    }
}

// Perform single-point crossover between two parents
void crossover(int parent1Index, int parent2Index) {
    for (int i = 0; i < POP_SIZE; i++) {
        int crossoverPoint = rand() % GEN_LENGTH;

        for (int j = 0; j < GEN_LENGTH; j++) {
            if (j < crossoverPoint) {
                newPopulation[i][j] = population[parent1Index][j];
            } else {
                newPopulation[i][j] = population[parent2Index][j];
            }
        }
        newPopulation[i][GEN_LENGTH] = '\0';
    }
}

// Apply mutation to the new population
void mutation() {
    for (int i = 0; i < POP_SIZE; i++) {
        for (int j = 0; j < GEN_LENGTH; j++) {
            if (((double)rand() / RAND_MAX) < MUTATION_RATE) {
                newPopulation[i][j] = 'A' + rand() % 26;
            }
        }
    }
}

// Replace the old population with the new population
void replacePopulation() {
    for (int i = 0; i < POP_SIZE; i++) {
        strcpy(population[i], newPopulation[i]);
    }
}

int main() {
    srand(time(NULL));
    int bestIndividualIndex;

    initializePopulation();

    for (int generation = 0; generation < 1000; generation++) {
        printf("Gen %2d :", generation + 1);
```

```
        bestIndividualIndex = evaluatePopulation();

        if (strcmp(population[bestIndividualIndex], target) == 0) {
            printf("Target tercapai di generasi ke-%d\n", generation + 1);
            break;
        }

        int parent1, parent2;
        selection(&parent1, &parent2);
        crossover(parent1, parent2);
        mutation();
        replacePopulation();
    }

    return 0;
}
```

Hasil :

```
PS D:\1. KULIAH\Semester IV\3. Sistem Cerdas\Demo Akhir\output> & .\'genetika_tugas.exe'
Gen  1 : 10 11 18 19  9 11 --- JKRSIK >> Fitness = 131
Gen  2 :  7 11 21 21 13 12 --- GKUUML >> Fitness = 134
Gen  3 :  7  1 18 21 11 12 --- GARUKL >> Fitness = 147
Gen  4 :  7  1 18 21 11 12 --- GARUKL >> Fitness = 147
Gen  5 :  3  1 18 21 11 12 --- CARUKL >> Fitness = 151
Gen  6 :  3  1 19 21 11  9 --- CASUKI >> Fitness = 155
Gen  7 :  3  1 19 21 11  9 --- CASUKI >> Fitness = 155
Gen  8 :  2  1 19 21 11  9 --- BASUKI >> Fitness = 156
Target tercapai di generasi ke-8
```

D. Analisa

Pada percobaan pertama, algoritma genetika diterapkan pada representasi individu dalam bentuk bilangan biner. Proses diawali dengan pembangkitan populasi awal yang terbentuk secara acak. Setiap individu kemudian dihitung nilai fitness-nya berdasarkan jumlah bit yang bernilai satu. Tahapan seleksi dilakukan menggunakan metode roulette wheel yang memberikan peluang lebih besar kepada individu dengan fitness yang lebih tinggi untuk terpilih menjadi induk pada generasi berikutnya. Proses crossover dan mutasi juga diterapkan untuk membentuk generasi baru yang lebih beragam, sehingga potensi untuk menemukan individu dengan fitness yang lebih baik semakin terbuka.

Dari hasil percobaan, terlihat bahwa meskipun individu dengan fitness maksimum sempat tercapai pada generasi awal, proses evolusi tetap dilanjutkan hingga beberapa generasi ke depan. Hal ini menunjukkan bahwa meski solusi optimal sudah ditemukan, proses seleksi, crossover, dan mutasi tetap berjalan untuk memperkaya variasi genetik dalam populasi. Setiap generasi menghasilkan individu baru dengan variasi bit yang beragam, memperlihatkan bagaimana mekanisme genetika bekerja dalam mencari solusi yang semakin baik melalui proses iteratif.

Sementara itu, pada percobaan kedua, algoritma genetika diterapkan untuk membentuk string yang sesuai dengan target tertentu, yaitu kata "BASUKI". Proses diawali dengan membangkitkan populasi string acak, kemudian dilanjutkan dengan seleksi individu terbaik berdasarkan fitness yang diukur dari kedekatan masing-masing huruf dengan target. Proses crossover dan mutasi diterapkan untuk menghasilkan generasi baru, hingga akhirnya individu yang identik dengan target berhasil ditemukan pada generasi ke8

E. Kesimpulan

Algoritma genetika mampu menyelesaikan permasalahan pencarian dan optimasi dengan pendekatan yang efektif melalui tahapan seleksi, crossover, dan mutasi yang berlangsung secara berulang. Pada percobaan pertama, algoritma ini berhasil menunjukkan bagaimana proses evolusi terus memperbaiki kualitas individu meskipun solusi terbaik sudah sempat ditemukan di generasi awal. Sementara itu, pada percobaan kedua, algoritma genetika terbukti mampu dengan cepat menemukan string target "BASUKI" hanya dalam delapan generasi. Hal ini menunjukkan bahwa algoritma genetika tidak hanya fleksibel untuk berbagai jenis masalah, tetapi juga mampu menemukan solusi yang diinginkan dengan cara yang alami dan bertahap.